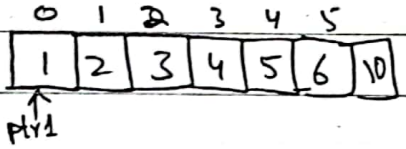
Day / Date

OOP LAB

Q1:

```
int getStringLength(char* str)
{
    if(*str == '\0')
    {
        return 0;
    }
    return 1 + getStringLength(str+1);
}
```

ptr1 = "123456"

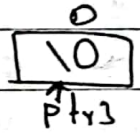


6	0
5	1 + gSL(ptr+1)
4	1 + gSL(ptr+1)
3	1 + gSL(ptr+1)
2	1 + gSL(ptr+1)
1	1 + gSL(ptr+1)
0	1 + gSL(ptr+1)

0+1 → 1 → 2 → 3 → 4 → 5 → 6

Ans

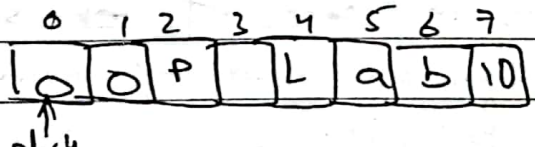
ptr3 = " "



0

Ans

ptr4 = "OOP Lab"



7	0
6	1 + g(p+1)
5	1 + g(p+1)
4	1 + g(p+1)
3	1 + g(p+1)
2	1 + g(p+1)
1	1 + g(p+1)
0	1 + g(p+1)

1 → 2 → 3 → 4 → 5 → 6 → 7

Day / Date

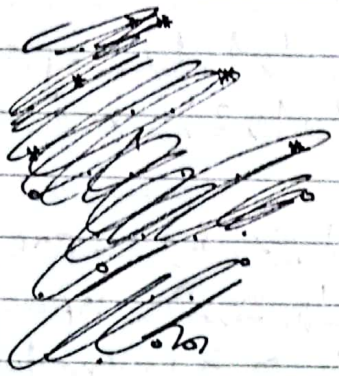
Loop:

```
int getStringLength(char *str)
{
    int count = 0;
    while(*str != '\0')
    {
        count++;
        str++;
    }
    return count;
}
```

Explanation:

A char pointer is like any pointer. It hold address and can be used for pointer arithmetic. A char pointer basically points to any character however it can also store strings as an array because a string is a character array in essence so a char pointer can be used to traverse strings like we traverse arrays with pointers. In case of ~~char~~ strings there is '\0' char at end of every string so we can check for it.

Day / Date



```
void printChar(char character, int count, int i = 1) {
{
    if (i > count) {
        return;
    }
    cout << character;
    printChar(character, count, iterator + 1);
}
void pattern(int n, int i = 0)
{
    if (i > count) {
        return;
    }
    printCharacter(' ', n - i - 1);
    cout << " *";
    printCharacter(' ', i * 2);
    cout << * << endl;

    pattern(n, * i + 1);
    printCharacter(' ', n - i - 1);
    cout << " 0";
    printCharacter(' ', i * 2);
    cout << "0" << endl;
}
```

Date

Line 1

pattern(3, 0)
printC(' ', 2, 1); space
printC(' ', 2, 2); space
printC(' ', 2, 3); return
*
printC(' ', 0, 1); return
* endl;

Line 2

pattern(3, 1)
printC(' ', 1, 1); space
printC(' ', 1, 2); return;
*
printC(' ', 2, 1); space
printC(' ', 2, 2); space
printC(' ', 2, 3); return;
* endl;

Line 3

pattern(3, 2)
printC(' ', 0, 1); return
*
printC(' ', 4, 1); space
printC(' ', 4, 2); space
printC(' ', 4, 3); space
printC(' ', 4, 4); space
printC(' ', 4, 5); return
* endl;

Line 4

pattern(3, 3) return;
printChar(' ');
pattern(3, 2)
printC(' ', 0, 1) i > c return
0
printC(' ', 4, 1) space
printC(' ', 4, 2) space
printC(' ', 4, 3) space
printC(' ', 4, 4) space
printC(' ', 4, 5) return;
0 endl;

Line 5

pattern(3, 1)
printC(' ', 1, 1) space
printC(' ', 1, 2) return
0
printC(' ', 2, 1); space
printC(' ', 2, 2); space
printC(' ', 2, 3); return
0 endl;

Line 6

pattern(3, 0)
printC(' ', 2, 1) space
printC(' ', 2, 2) space
printC(' ', 2, 3) return
0
printC(' ', 0, 1) return
0 endl;

Day / Date

In this code, the backtracking mechanism is shown in the way the function prints the bottom half of the diamond. For top half it is just simple forward recursion however we should keep in mind that these recursive calls are still loaded into memory. As soon as the iterator >= count condition is true, these function start returning in reverse order e.g:

```
pattern(3,0)   ]
pattern(3,1)   ] Forward recursion
pattern(3,2)   ]
pattern(3,3)   condition true, return
pattern(3,2)   ]
pattern(3,1)   ] Backtracked calls.
pattern(3,0)   ]
```

Difference:

- In my code, I basically used the helper function to print the spaces and in the solution helper is used to do the same.
- The logic is basically the same in both codes except in my helper I am using one argument 'line' to keep track of columns for spaces, and my iterator is initialized with 1 row instead of 0 but it balances out in the conditions.

Day / Date

Q4(a) Method overloading is a type of ~~early binding~~ compile-time polymorphism (early binding) where multiple methods in the same scope have the same name but have a difference in parameter lists. This allows them to adjust their task depending on parameters.

Method overriding is a type of run-time polymorphism (late binding) where a derived class provides a specific implementation of a method that is already defined in its base class. This is achieved using the 'virtual' keyword in C++ to allow derived class method to override base class method using pointer types.

(b) OOP uses objects to implement programs. A class is a blueprint from which objects are created and it defines a class by it's data members and methods. An object is the instance of a class / implementation of the class.

(c) Polymorphism basically means "many shapes" and it allows objects of different classes to be used as objects of a common base class. In OOP, polymorphism is used for Compile Time Polymorphism And Run time Polymorphism.

Main benefit of polymorphism is that it promotes flexibility and the ability to define methods which can work with objects differently depending on their data type / class which promotes code reusability and maintainability.

Day / Date

(d) Compile Time Polymorphism, also known as early binding, makes use of method overloading and operator overloading. It is determined at compile time.

Run time Polymorphism, also known as late binding, is done through method overriding. This allows a method to be defined in a base class and derived class. The determination of which method to use is done at run time based on type of pointer.

(e) Inheritance: This allows a heirarchy between classes of vehicles. A base class 'Vehicle' can define common attributes and methods such as bookRide() and derived vehicles classes like 'Bike' or 'ACCar' can inherit these common functionalities and attributes.

Polymorphism: This can be seen in functions like calcRate(), where a base class 'Vehicle' can have a pure virtual 'calcRate()' function to account for different pricing.

Day / Date

(f) For Brick Breaker, the best approach would depend on the specific requirements of the game.

- Make one paddle class: This approach uses composition and is better if the paddles share most behaviors and attributes.

- Make a base and derived class: This approach would be useful only if the paddles had a very significant difference i.e different player scores counters for both paddles.

As the first case applies in this project, it is the approach I used.

(g) A "has-a" relationship denotes composition, where a class contains the object of another class. It denotes a strong relationship e.g if parent class is destroyed, child class may also be destroyed.

An "is-a" relationship denotes inheritance, where a class is the extension of another. It signifies a heirarchal relationship. Inheritance allows extending a base class e.g a "Dog" is an "Animal".

Day / Date

(h) Encapsulation is the process of bundling data and methods into a single class and restricting access to some components of the object. This is done using access modifiers like private, public and protected to control visibility of class members.

Abstraction is the concept of hiding the complex details and showing only essential features of the object. It is basically simplification of complex systems by exposing only what is necessary. It is achieved through abstract classes and virtual/pure virtual functions.

Q5:

```
Matrix(const Matrix& other){ //Copy Constructor
        allocateMatrix(other.rows, other.cols);
        for(int i=0; i<rows; i++){
            for(int j=0; j<cols; j++){
                matrix[i][j] = other.matrix[i][j];
            }
        }
}
Matrix operator*(const Matrix& other) const {

    if(cols != other.rows){
        cout<<"Invalid\n";
        return *this;
    }
```

```
        Matrix result (rows, other.cols);
        for(int i=0; i< rows; i++) {
        { for(int j=0; j< cols; j++) { result.matrix
            for(int k=0; k < cols; k++){ [i][j] =0;
                result.matrix[i][j] += matrix[i][k] *
                    other.matrix [k][j];
            }
        }
        }
        return result;
}

Matrix& operator = (const Matrix& other) {
    if ( this == &other) {
        return *this;
    }
    deallocateMatrix();
    allocateMatrix( other.rows , other. cols);
    for (int i=0; i< rows; i++) {
        for (int j=0; j< cols; j++) {
            matrix [i] [j] = other.matrix [i] [j];
        }
    }
    return *this;
}
```

Day / Date

Difference between = operator and copy constructor:

A copy constructor is used when a new object is created from an existing object i.e:

- When an object is passed by val
- When an object is returned by val
- When an object is copied during creation e.g Matrix matrix2 = matrix1;

The equals operator is used when we are assigning the value of an existing object to another e.g matrix3 = matrix2;

Q6: Polymorphism is a fundamental concept in OOP that allows objects of different types to be created as objects of a common base type. It lets different implementation of methods of the same name in a heirarchal structure / inheritance structure. This can be done using ~~overrun~~ runtime polymorphism. Compile time polymorphism can be used in same scope structures.

Example of Polymorphism:

ay / Date

```
class Account {
    protected:
        float balance;
        int deposits;
        int withdrawals;
        float ir;
        float msc;

    public:
        Account(float balance, float ir) : balance(balance)
, ir(ir), deposits(0), withdrawals(0), msc(0) {}

        virtual void deposit(float input){
            if (input > 0) {
                balance += input;
                deposits ++;
            }
            else {
                cout<<" Enter a valid amount. \n";
                return;
            }
        }
};
```

ay / Date

```
class SavingsAccount : protected Account
{
    bool status;

    public:
        SavingsAccount (float balance, float ir) :
        Account (balance, ir) {
            if (balance < 25000) {
                status = false;
            }
            else {
                status = true;
            }
        }

    void deposit (float input) {
        if (status == false) {
            if (input + balance > 25000) {
                status = true;
            }
        }
        Account::deposit (input);
    }
};
```

Day / Date

Q7:

```
class Order {

    private:
        Customer customer;
        struct OrderItem {
            Product product;
            int quantity;

            OrderItem() {}
            OrderItem(const Product product, int
quantity): product(product), quantity(quantity) {}
        };
        OrderItem *items;
        int numberOfItems;
        int itemCount;

    public:

        Order(const Customer& customer, int numberOfItems):
customer(customer), numberOfItems(numberOfItems) {
            itemCount = 0;
            items = new OrderItem[numberOfItems];
        }

    ~Order() {

        delete[] items;
    }
```

Date

Q8:

```
void printLine(int d, int count, char character, int i=1){

    if(i>d) {
        return;
    }
    if(i<=count || i>dimension - count) {
        cout<<character;
    } else {
        cout<<" ";
    }
    printLine(d, count, character, i+1);
}
```

Pixel

CamScanner

Date _____

For number = 3;

Line 1

```
pattern1(3, 1);
    printLine(3, 1, *);      *
    printLine(3, 1, *, 2);   space
    printLine(3, 1, *, 3);   *
    printLine(3, 1, *, 4); return.

    printLine(3, 1, o);      o
    printLine(3, 1, o, 2);   space
    printLine(3, 1, o, 3);   o
    printLine(3, 1, o, 4); return
```

Line 2

```
pattern1(3, 2);
    printLine(3, 2, *);      *
    printLine(3, 2, *, 2);   *
    printLine(3, 2, *, 3);   *
    printLine(3, 2, *, 4); return;

    printLine(3, 2, o);      o
    printLine(3, 2, o, 2);   o
    printLine(3, 2, o, 3);   o
    printLine(3, 2, o, 4); return;
```

Forward Recursion

Backtracking

Line 3

```
pattern1(3, 2)
    printLine(3, 2, o);      o
    printLine(3, 2, o, 2);   o
    printLine(3, 2, o, 3);   o
    printLine(3, 2, o, 4) return.

    printLine(3, 2, *);      *
    printLine(3, 2, *, 2);   *
    printLine(3, 2, *, 3);   *
    printLine(3, 2, *, 4); return.
```

Line 4

```
pattern1(3, 1)
    printLine(3, 1, *);      *
    printLine(3, 1, *, 2);   space
    printLine(3, 1, *, 3);   *
    printLine(3, 1, *, 4); return

    printLine(3, 1, o);      o
    printLine(3, 1, o, 2);   space
    printLine(3, 1, o, 3);   o
    printLine(3, 1, o, 4); return.
```

```
* . * o . o
* * * o o o
o o o * * *
o . o * . *
```

Date _____

Iterative Solution:

```
void printLine(int dimension, int stericCount, char character)
{
    for(int i=1; i<=dimension; i++)
    {
        if(i<=stericCount || i>dimension-stericCount)
        {
            cout<<character;
        }
        else
        {
            cout<<" ";
        }
    }
}
void pattern1(const int dimension)
{
    for(int i=1; i<=(dimension/2)+1; i++)
    {
        printLine(dimension, i, "*");
        printLine(dimension, i, "o");
        cout<<endl;
    }
    for(int i=(dimension/2)+1; i++)
    {
        printLine(dimension, i, "o");
        printLine(dimension, i, "*");
        cout<<endl;
    }
}
```

Pixel

CamScanner

Date _____

Backtracking:

In the recursive solution, backtracking is used to basically copy the upper pattern but in reverse. The function creation / deletion is as follows:

pattern1(3,1) created * * * *
pattern1(3,2) created * * * * * *
pattern1(3,3) created] does nothing as immediate
pattern1(3,3) destroyed] return.
pattern(3,2) destroyed * * * * * *
pattern(3,1) destroyed * * * *

Backtracking is basically restoring the ~~function~~ program to a previous state after the ~~middle~~ last function creation is done. Example:

```
1  printLine(d, i, 'o')
2  cout << endl;
3  pattern1(d, i+1);
4  printLine(d, i, 'o');
```

As soon as innermost function returns, we move to second innermost and run Line 4 onwards till end of scope and this process is followed until first function call returns;

Pixel

CamScanner